IE University

School of Science & Technology

Zerocash

A Review of Decentralized Anonymous Payments from Bitcoin

Group 4

Andrei Baraitaru, Clara Mouzannar, Gloria Paraschivoiu, Massimo Ridella, Zaid Alsaheb

Bachelor of Computer Science & Artificial Intelligence

Supervised by:

Milan Groshev

School of Science & Technology, IE University, Spain

Blockchain Group Assignment

1 Introduction

Bitcoin is the first digital currency to see widespread adoption. Unlike earlier e-cash schemes (Camenisch et al., 2005; Chaum, 1982), Bitcoin requires no trusted parties. Instead of a bank, Bitcoin uses a ledger known as "Blockchain" to keep record of user transactions. In an attempt to anonymise these records, users often employ many identities or *pseudonyms*; however, research has shown how anyone can *de-anonymize* transactions by using publicly available information in the blockchain.

Consequently, users motivated enough to maintain their transactions private use *mixes* to blur their transaction history. The user sends a set of coins to a trusted party (the *mix*), and after a time interval receives the same amount back in different coins. Mixes are however high risk for the following reasons:

1. The delay to reclaim coins must be large to allow enough coins to be mixed in.
2. The mix operator can trace coins.
3. The mix operator may steal coins.

Whilst users with "something to hide" are willing to take these risks, common users who care about their privacy do not. Not only is it in the user's interest to anonymise transactions, it is also in the currency's interest. Anonymising a coin's history helps ensure the coin remains fungible. By tracking a coin's history, an observer could claim it as *tainted* (used in a crime) and refuse it, or *special* and claim the value is higher (non-fungible).

These risks drove the development of Zerocoin (Miers et al., 2013). Zerocoin relies on zero-knowledge proofs to provide strong anonymity. Zerocoin acts as a trustless *mixer* built on top of Bitcoin: coins are authenticated by proving, in zero-knowledge, that they belong to a public list of valid coins stored on the blockchain. Users can regularly "wash" their coins using the Zerocoin protocol and do not rely on a trusted entity.

Research recommends, however, to run day-to-day transactions on Bitcoin. Zerocoin transactions are more expensive and heavy in computation compared to normal Bitcoin transactions. The costs mostly originate from the zero-knowledge approach, which requires heavy

computation and network burden. Furthermore, Zerocoin was not designed to be a currency in itself; one user cannot pay another directly in “zerocoins”. Finally, Zerocoin makes transactions anonymous by unlinking a coin from its origin only; it does not hide the amount or other metadata about the transaction. Zerocash (Ben-Sasson et al., 2014) addresses each of these limitations.

1.1 Related Work

A vast amount of research has been done on anonymous e-cash. Bitcoin represented a new direction toward decentralised digital currency. Instead of relying on a single trusted bank, Bitcoin uses a decentralised ledger called the Blockchain. The Blockchain allows for peer-to-peer transactions without needing a trusted bank. Even though Bitcoin introduced many innovations toward decentralised digital currency, it did not achieve strong privacy. Due to this limitation, researchers began developing privacy-enhanced methods within the Bitcoin ecosystem.

While mixing services do enhance user privacy, as previously mentioned, they have several disadvantages. Decentralised alternatives to mixing services include CoinJoin. CoinJoin attempts to eliminate the need for a third party by coordinating transactions between multiple users. Unfortunately, CoinJoin suffers from scalability problems, high overhead for coordination, and is vulnerable to denial-of-service attacks.

Zerocoin (Miers et al., 2013) represents a significant enhancement over existing solutions because it introduces a cryptographic protocol that serves as a decentralised mixer. Zerocoin is built as an extension to Bitcoin and allows users to transform bitcoins into anonymous coins and then redeem them at a future time without any relation to how the original transaction occurred. However, Zerocoin has its own limitations. Transaction processing is computationally intensive, requiring large proofs that must be verified and retained by the network. Furthermore, Zerocoin utilises fixed denominations, does not allow users to directly pay each other, and does not conceal either the transaction value or its metadata.

To address the above-mentioned limitations, recent research has focused on increasing both the performance and the functionality of anonymous cryptocurrencies. Zerocash (Ben-

Sasson et al., 2014) builds on the concepts of Zerocoin by employing advanced cryptographic primitives known as zk-SNARKs (Zero-Knowledge Succinct Non-Interactive Arguments of Knowledge). zk-SNARKs enable the creation of dramatically smaller proofs and significantly speed up the process of verifying them, thereby providing better performance characteristics than anonymous transactions had previously experienced. Importantly, Zerocash enables fully private transactions where neither the source nor the destination of funds are revealed, along with concealing the transaction value.

In comparison to previous solutions, Zerocash addresses the usability concerns associated with Zerocoin. It allows for direct payments between users and supports variable transaction values, addressing two of the most significant usability shortfalls of its predecessor. Similar to previous solutions, Zerocash operates on a public ledger without requiring the presence of a trusted entity, except for an initial setup phase. However, Zerocash achieves this improvement at the cost of additional computational intensity during proof generation.

2 Technical Contribution and Methodology

2.1 Technical Contribution

One of the most significant contributions of the Zerocash paper is the protocols, the mathematics and the algorithms behind its validation process. Prior work, including Zerocoin (Miers et al., 2013), allowed users and their transactions to be anonymous through informal arguments and descriptions, but without a definition of what an "anonymous payment" actually means in a decentralised setting. In Section 3 of the paper, this is solved by the introduction of a *Decentralized Anonymous Payment* (DAP) scheme, where formal cryptography defines both the requirements and the security properties that any construction must satisfy to be considered valid.

This is important because, firstly, it makes the paper's claims falsifiable when it comes to security: the authors can prove, under specific cryptographic assumptions, that their construction satisfies the definition. Secondly, it establishes a reusable framework that future work can build upon or challenge. The decision to separate the definition (Section 3) from the construction (Section 4) reflects sound methodology and is a strength that differentiates the paper from more engineering-oriented blockchain work of the same period.

The DAP scheme is defined as a tuple of six polynomial-time algorithms $\Pi = (\text{Setup}, \text{CreateAddress}, \text{Mint}, \text{Pour}, \text{VerifyTransaction}, \text{Receive})$. Each captures a distinct aspect of an anonymous currency:

- **Setup** performs a one-time generation of public parameters (pp); no secrets are kept after this step, which bounds the window of trust to a single event.
- **CreateAddress** generates a key pair (addr_{pk}, addr_{sk}); a user may create as many of these as they wish, allowing for pseudonyms.
- **Mint** converts v units of the base currency (for example, Bitcoin) into an anonymous coin c, recording only a coin commitment cm on the ledger.

- **Pour** is the central operation. It consumes two input coins and produces two output coins while hiding all private values (sender, receiver and amount), while also optionally declaring a public output v_{pub}.
- **VerifyTransaction** validates transactions for inclusion in the ledger.
- **Receive** allows a user to scan the ledger and recover coins directed to their address by trial-decrypting the ciphertexts embedded in pour transactions.

This is a modular design that works well for decentralised deployment: validation (VerifyTransaction) is cheap and public, while the expensive operations (Mint, Pour) are borne only by the transacting parties.

The ledger model introduces two new transaction types alongside existing Basecoin transactions. A mint transaction $\text{tx}_{\text{Mint}} = (\text{cm}, v, *)$ records the deposit of value v under commitment cm. A pour transaction $\text{tx}_{\text{Pour}} = (\text{rt}, \text{sn}^{\text{old}}_1, \text{sn}^{\text{old}}_2, \text{cm}^{\text{new}}_1, \text{cm}^{\text{new}}_2, v_{\text{pub}}, \text{info}, *)$ records the consumption of two coins (by serial number) and the creation of two new ones (by commitments), with only v_{pub} and info visible. Two auxiliary data structures are maintained: CMList, the list of all coin commitments appearing in mint and pour transactions, stored as a Merkle tree with root rt; and SNList, the list of all revealed serial numbers used for double-spend detection. The Merkle tree enables logarithmic-time membership proofs, which is needed for the zk-SNARK's efficiency since the proof only needs to check a path from a leaf to the root rather than show all commitments. Importantly, all past Merkle roots are retained in the ledger, so a coin committed under an older state can still be spent without requiring the prover to use the most current root.

The paper designs security through three game-based definitions, each with a distinct threat model. **Ledger Indistinguishability** (L-IND) requires the ledger to reveal no information beyond what is already public: no computationally bounded attacker can distinguish between two ledgers that differ only in private data, as long as their public behaviour is the same. This captures anonymity of the sender, the recipient, and the amount, which is stronger than Zerocoin (where only the sender was hidden). **Transaction Non-Malleability** (TR-NM) requires that no adversary can take a valid pour transaction and produce a modified version that shares a serial

number yet validates differently. Without this property, an attacker intercepting a transaction before it is confirmed could redirect the public output v_{pub} to a different address, effectively stealing the payment destination. This is a non-trivial concern in Bitcoin-style systems where transactions travel through untrusted nodes before being confirmed. Finally, **Balance** requires that no adversary can spend more than they have legitimately minted or received. This prevents an attacker from forging coins or double-spending to gain value they never legitimately owned. The game tracks five value quantities and ensures their accounting is consistent.

For a critical evaluation of Section 3, the formal model is a genuine contribution but with limitations worth pointing out. The most important is the trusted setup: if the zk-SNARK trapdoor leaks, an adversary can forge proofs and mint money from nothing. L-IND survives a bad setup but Balance does not, which is a real constraint. The authors suggest a multi-party computation as a remedy but leave it as future work. The 2-input/2-output constraint on Pour is justified in theory through composition, but in practice chaining many pours for complex transactions adds significant time and overhead. Finally, the Receive algorithm has to scan and trial-decrypt every ciphertext on the ledger; selective scanning would itself leak privacy, so this is unavoidable but becomes increasingly costly as the ledger grows.

2.2 Methodology

To actually build the DAP scheme, Section 4 of the paper relies on five cryptographic tools working together. The first is a *collision-resistant hash function*, on which the Merkle tree of coin commitments is built. The second is a *pseudorandom function*, used to generate address keys and serial numbers; it produces three separate functions from a single seed by using different prefixes, which keeps each purpose cleanly separated. The third is a *commitment scheme* that statistically hides coin contents: this is a stronger form of hiding than the usual computational version, and is what makes long-term anonymity possible even if certain assumptions are later broken. The fourth is a *one-time signature scheme* that ties all the fields of a pour transaction together so that nothing in the transaction can be altered after it is created. The fifth is an *encryption scheme with key-privacy*: normal encryption hides what is being said, but

key-private encryption also hides who it is being said to, which matters because without it an observer could link ciphertexts on the ledger back to specific recipients.

The core of Section 4 is the POUR statement. This is essentially a checklist of everything that needs to be true for a pour transaction to be valid. The clever part is that, instead of checking these conditions by revealing all the private details, the system uses a zk-SNARK proof that confirms everything is correct without showing anything private.

Concretely, when someone spends a coin, the proof needs to show six things:

1. The coins being spent actually exist on the ledger; their commitments appear in the Merkle tree.
2. The spender actually owns those coins; their secret key matches the public key attached to the coin.
3. The serial numbers are correctly generated and not forged.
4. All four coins involved (the two being spent and the two being created) are properly formed.
5. The anti-tampering values h_1 and h_2 are correctly computed, which ties into the non-malleability property.
6. The values balance: what goes in equals what comes out plus any public payment.

All of this is confirmed by the proof without revealing any of the actual values, keys, or which specific coins on the ledger are being spent. This is exactly what makes ledger indistinguishability work in practice.

One design detail worth understanding is how coin commitments are structured. The system needs to solve a tricky tension: anyone should be able to verify that a coin has a certain value, but nobody should be able to see who owns it or what its serial number is. The solution is a two-phase commitment. First, the address key and serial number seed are locked inside an inner commitment k. Then, the coin value and that inner commitment are locked together into the outer commitment cm, which is what actually appears on the ledger. The result is that anyone can verify the coin's value by checking the outer commitment (the mint transaction publicly reveals enough information to do exactly that), but the owner's identity and serial number are

hidden inside k, which is itself a statistically-hiding commitment, meaning they stay private even to someone who can see everything on the ledger. It is a neat two-layer design that solves both requirements at once without compromising either.

The construction is sound and the security proof is well structured, but several points are worth questioning. First, the **strength of the assumptions**: the zk-SNARK relies on knowledge-of-exponent assumptions, which are considered non-falsifiable, meaning there is no experiment that could disprove them even if they were actually false. This is a stronger and less standard type of assumption than what most systems rely on, and while the authors acknowledge it, it is a real caveat for anyone evaluating how trustworthy the system is. Second, **proving cost**: generating a pour transaction takes around two minutes on a normal machine because the zk-SNARK has to verify a circuit with over four million gates, mostly coming from SHA-256 computations inside the proof. Verification is fast at under six milliseconds, which is fine for network nodes, but the two-minute proving time is a genuine problem for any use case involving quick or frequent payments. Third, the **fixed Merkle tree depth**: the circuit is hardcoded to support up to 2^{64} coins, which means the proving key size and proving time are fixed regardless of how many coins actually exist; a small system pays the same cost as a massive one, which is an inefficiency. Fourth, the **gap between the formal model and practice**: the paper handles complex transactions by chaining multiple pours together, but this chaining mechanism is never formally modelled in the security definitions of Section 3, so the security guarantees technically only cover the simple 2-input/2-output case rather than realistic multi-pour usage.

2.3 Integration with Existing Ledger-based Currencies

A key part of the paper describes how the protocol can be implemented in practice on top of existing ledger-based systems. The authors note that Zerocash is compatible with any ledger, such as Bitcoin or its forks, but it adds new transaction types and semantics that do not interoperate with the existing Bitcoin network. Although integrating into Bitcoin is technically feasible with a coordinated upgrade, the authors do not suggest it in the near term and instead propose deployment on a separate altcoin or modified system.

Two main integration approaches are proposed. The first is to completely switch to Zerocash as the base currency. In this model, all transactions are carried out under the DAP scheme, and payments, minting and transfers are anonymous by default. This offers very high privacy guarantees but at a cost: every transaction is required to construct a zk-SNARK proof, existing Bitcoin capabilities such as scripting are lost, and transactions must be confirmed before they can be spent, eliminating the flexibility of unconfirmed transactions.

The second is a hybrid currency, in which zerocoins exist on the same ledger as regular bitcoins. Users can convert between them at a one-to-one rate. Mint transactions burn bitcoins and produce zerocoins, in effect substituting public coins with private ones. Pour transactions can then transfer zerocoins using zero-knowledge proofs rather than explicit inputs, and may declare a public output v_{pub}.

Under the hybrid system there are two approaches to accounting. Bitcoin operations can still be publicly verified, ensuring inputs match outputs and fees. Zerocoin transactions are authenticated privately under the DAP scheme, ensuring validity without disclosing transaction details. The protocol also enables multiple pour operations to be combined into a single transaction, increasing flexibility and efficiency.

To support these features, the Bitcoin protocol must be extended. The paper proposes either compiling hard-coded validation of Zerocash transactions into nodes or adding a new opcode to the scripting language that calls VerifyTransaction. In both approaches, global data structures are only updated after block acceptance.

Lastly, the authors point out that only the transaction ledger is anonymised in Zerocash. User identities can still be leaked through network-level information such as IP addresses or timing patterns. They suggest mitigating this with anonymity networks like Tor or mixnets. They also describe a “poison-pill” attack in which a manipulated block could be used to identify a user, and propose as a defence that users verify their view of the blockchain against trusted peers.

3 Experiments

The Zerocash paper evaluates the practicality of its DAP scheme through three sets of experiments: micro-benchmarks of the underlying zk-SNARK for the POUR statement (Section 7.1 of the paper), micro-benchmarks of the six DAP scheme algorithms (Section 7.2), and a 1,000-node network simulation testing the impact of Zerocash transactions on Bitcoin-style network propagation (Section 7.3). All measurements are taken at the 128-bit security level, with results averaged over ten runs and standard deviations under 2.5%.

The micro-benchmarks were performed on two machines: an Intel Core i7-2620M laptop (2.70 GHz, 12 GB RAM) and an Intel Core i7-4770 desktop (3.40 GHz, 16 GB RAM), with both single-thread and multi-thread (four-thread) configurations on the desktop. The arithmetic circuit underlying the POUR statement was hand-optimised to exploit non-deterministic verification and the field characteristic, and all cryptographic primitives were instantiated using SHA-256 to keep the circuit small.

The network simulation used 200 Amazon EC2 `m1.medium` instances running 1,000 Bitcoin nodes between them, in a single AWS region on a private network. Transactions were generated as a Poisson process at a target rate of one per second, and blocks at every 150 seconds (matching Litecoin's block interval rather than Bitcoin's, to keep simulations short). Each simulation run lasted one hour. The proportion ε of Zerocash transactions in the traffic mix was varied across $\{0\%, 25\%, 50\%, 75\%, 100\%\}$. Three metrics were measured: transaction latency from creation to inclusion in a block, block propagation time across the network, and per-block verification time. The simulation design isolates the impact of zk-SNARK verification cost from the rest of Bitcoin's processing pipeline.

4 Results

4.1 zk-SNARK Performance

The zk-SNARK is the cryptographic engine of Zerocash. It is invoked once per spend transaction (a Pour) and verified by every node on the network. Performance across the two test machines and two threading configurations is summarised in Table 1.

		Intel Core i7-2620M @ 2.70 GHz 12 GB of RAM	Intel Core i7-4770 @ 3.40 GHz 16 GB of RAM	
		1 thread	1 thread	4 threads
KeyGen	Time	7 min 48 s	5 min 11 s	1 min 47 s
	Proving key	896 MiB		
	Verification key	749 B		
Prove	Time	2 min 55 s	1 min 59 s	46 s
	Proof	288 B		
Verify	Time	8.5 ms	5.4 ms	

Table 1: Performance of our zk-SNARK for the NP statement POUR ($N = 10$, $\sigma \leq 2.5\%$).

Two structural results stand out. First, every Zerocash proof has a **constant size of 288 bytes**, regardless of how many coins exist in the system or how complex the underlying statement is. Second, **verification time is essentially constant in milliseconds**, which is the property that determines whether the scheme is deployable, since every node verifies every transaction. Key generation is the most expensive operation but is performed exactly once at system setup, producing an 896 MiB proving key and a 749 B verification key.

4.2 DAP Scheme Performance

The end-to-end Zerocash protocol exposes six algorithms. Their measured costs on the desktop machine are:

- **Setup**: 5 minutes (one-time, dominated by key generation).
- **CreateAddress**: 326 ms.
- **Mint**: 23 µs, producing a coin of 463 B and a mint transaction of 72 B.

- **Pour**: 2 minutes, producing a pour transaction of 996 B + |info|.
- **VerifyTransaction**: 8.3 μs for a mint, 5.7 ms for a pour.
- **Receive**: 1.6 ms per pour transaction scanned.

The asymmetry between Pour (∼2 minutes) and VerifyTransaction (5.7 ms) is intentional: the cost is borne by the transacting party, not by the network. Because the proof size is constant and verification is fast, network nodes see Zerocash transactions as roughly equivalent in cost to plain Bitcoin transactions.

4.3 Large-Scale Network Simulation

To test whether the per-transaction costs translate to a viable network, the authors ran a 1,000-node Bitcoin testnet on 200 Amazon EC2 instances, with 150 s block times and varying the proportion ε of Zerocash transactions from 0% to 100%. They measured three metrics:

- **Transaction latency** (creation to inclusion in a block): 95 s at low Zerocash density, rising to 190 s at 100% Zerocash traffic.
- **Block propagation time**: between 2 and 4.5 s depending on traffic mix; the relationship to ε is not strictly monotonic.
- **Block verification time**: under 80 ms in the worst case, far below the theoretical maximum of 1,500 ms predicted by the per-transaction cost.

The fact that block verification time stayed two orders of magnitude below the worst-case prediction is explained by Bitcoin's verification caching: a transaction that has already been seen propagating through the network does not need to be re-verified when it appears in a block. Caching turns out to be effective even when 100% of network traffic is Zerocash.

4.4 Comparison with Zerocoin

Relative to the prior state of the art (Zerocoin, Miers et al. 2013), Zerocash reports:

- **Transaction size**: from 45 kB down to under 1 kB, a reduction of over 97.7%.
- **Verification time**: from 450 ms down to under 6 ms, a reduction of over 98.6%.

It also adds capabilities Zerocoin lacked: variable-amount transactions, hidden transaction values, hidden coin balances, and direct user-to-user payments without a Bitcoin fallback.

5 Discussion

5.1 Interpretation of the Results

These results matter because they cross the practicality threshold that prior anonymous payment proposals could not. The deployability question for any privacy-preserving cryptocurrency reduces to a single concern: does adding privacy break the network's ability to validate transactions in time? Zerocash answers this in two parts.

The prover-side cost (2 minutes per Pour, 5 minutes per Setup) is significant but acceptable. It runs on the spender's machine, in private, exactly once per transaction. A user who is willing to wait two minutes for privacy can have it. Importantly, this cost does not scale with the number of coins in the system; it is bounded by the size of a fixed arithmetic circuit.

The verifier-side cost is the more important number, and it is small. Verification takes 5.7 ms for a Pour transaction, with a constant proof size of 288 bytes. This means that as the Zerocash anonymity set grows from one user to a million users, the cost of validating a transaction does not change. Compared with Zerocoin's 450 ms and 45 kB, the difference is not incremental but categorical: Zerocoin would saturate the network's capacity at a small fraction of Bitcoin's transaction rate, whereas Zerocash transactions are competitive with plain Bitcoin transactions.

The simulation results reinforce this. Even in the worst-case scenario where 100% of network traffic was Zerocash, block verification time stayed below 80 ms, well within Bitcoin's tolerance, and block propagation increased only modestly. Verification caching (where a node skips re-verifying a transaction it has already seen propagated) is effective in practice because most transactions are seen before they are confirmed in a block. The fact that latency rises only from 95 s to 190 s at full Zerocash density tells us the bottleneck is the Bitcoin block interval itself, not the SNARK verification.

5.2 Limitations

Despite these results, several limitations are acknowledged in the paper, and others follow from its assumptions.

1. **Trusted setup.** The zk-SNARK construction requires a one-time trusted setup that produces public parameters. The soundness of every subsequent proof depends on the secret randomness used in this setup being destroyed. If a setup participant retains the secret, they can in principle forge proofs and counterfeit money, though, importantly, the anonymity property still holds even against a corrupt setup. This is the most cited weakness of the construction and a known structural property of pre-processing zk-SNARKs.
2. **Computational rather than statistical anonymity.** The default construction provides only computational privacy, meaning anonymity holds against a computationally bounded adversary. An unbounded adversary, for example one with future quantum computing capability, could in principle decrypt the ciphertexts attached to pour transactions and brute-force the senders and recipients. The paper notes this and proposes an everlasting-anonymity variant (Section 8.1) at the cost of an out-of-band communication channel.
3. **Network-level deanonymisation.** Zerocash anonymises only the transaction ledger. Network metadata, such as IP addresses, timing patterns, and peer connections, still leaks identifying information. The paper recommends layering Tor or a Mixnet on top, but this is left as an external dependency rather than solved within the protocol itself.
4. **The "poison-pill" block attack.** A powerful adversary capable of producing valid blocks could craft a block specifically targeted at one user. If the user is the only one to spend coins relative to that block's Merkle root, they uniquely identify themselves. Mitigation requires comparing one's view of the chain with trusted peers, which weakens the trustless model.
5. **Storage growth.** Two data structures grow without bound: CMList (all coin commitments) and SNList (all spent serial numbers). Bitcoin nodes can prune unspent-output sets, but Zerocash nodes must retain the full SNList to detect double-spends. The paper observes this could become prohibitive at scale and proposes mitigations in Section 8.3.
6. **Receive cost scales with traffic.** Detecting incoming payments requires a recipient to trial-decrypt every pour-transaction ciphertext on the chain, at 1.6 ms each. In a high-volume

network this dominates wallet costs and may force reliance on out-of-band notification channels, which themselves leak metadata.

7. **Prover-side latency.** Two minutes to produce a Pour, and five minutes for system setup, is an order of magnitude slower than plain Bitcoin transaction creation. While acceptable for desktop use, this rules out latency-sensitive applications and was, at the time of publication, infeasible on mobile hardware.
8. **Simulation realism.** The 1,000-node simulation is approximately 1/16 the size of the reachable Bitcoin network. The authors used a simplified proof-of-work, ran on shared EC2 hardware that introduced timing variability, and saw their target transaction rate slip from 1/s to roughly 1/1.4 s. The results are directionally informative but should not be read as a tight bound on production behaviour.
9. **Compliance and oversight tension.** Strong unconditional privacy is in tension with regulatory regimes that depend on transaction visibility. The paper's conclusion frames this as a research-and-policy question rather than a solved problem.

5.3 Impact on Blockchain Evolution

The goal of the paper was to fix the problems with Zerocoin in a more efficient and private way. Instead, it became much more. Zerocash was not just an improvement over an existing technology; it changed the way people think about what is possible on the blockchain. The implications of that change can be felt today in every corner of the cryptocurrency world, from Ethereum's ability to process millions of transactions to the way that governments write financial regulation.

5.3.1 The Paper That Launched a Cryptocurrency

The paper's most immediate result was the launch of Zcash, a working cryptocurrency based very closely on the Zerocash protocol, in October 2016. Several of the paper's authors went on to found the Electric Coin Company, the organisation responsible for the development of Zcash. Zcash was one of the earliest privacy-preserving cryptocurrencies, and at one point had a multi-billion-dollar market capitalisation. The most uncomfortable truth that the team had

to face was that the underlying cryptography required a trusted setup, a ceremony to generate and destroy some secret randomness. If one participant were to keep a copy of the secret key, they could print endless counterfeit money and no one could catch them. To address this, the Zcash team performed a multi-party ceremony in which six people in various locations each contributed a portion of randomness. Since the system theoretically requires only at least one participant to have destroyed their secret for the system to remain secure, the ceremony became something of an event in the cryptography community, and was reported to have ended with one participant destroying their equipment.

A second effect, which is more subtle but turned out to be enormously important, is that the authors decided to perform everything using SHA-256 so that their cryptographic circuit would be small. This made explicit something that had never been so closely articulated before: the choice of hash function is the single biggest cost driver in a zero-knowledge proof. This observation directly inspired the entire subfield of "SNARK-friendly" hash functions (including MiMC in 2016 and Poseidon in 2019), yielding protocol designs that are substantially cheaper to prove and leading to the inclusion of these hash functions in practically every modern ZK application.

5.3.2 How a Privacy Tool Became a Scalability Tool

The surprising part of the story is that the same technology Zerocash created to hide transactions is now the primary technique Ethereum uses to process more transactions. The observation we saw in the paper, that you can take an arbitrarily complicated computation and boil it down to a succinct proof that is very fast to verify, turns out to be just as useful for scalability as it is for privacy, and is the basis for one of the most important blockchain scaling technologies, ZK-Rollups. A ZK-Rollup takes thousands of transactions, processes them off the main Ethereum chain, and posts a single cryptographic proof to the Ethereum chain stating "all of these transactions were valid". Every node can verify this proof within milliseconds no matter how many transactions the proof represents, a huge saving in fees and congestion. Today, zk-Rollups like zkSync, StarkNet, and Polygon zkEVM are in production and handle a significant

share of Ethereum's throughput, yet none of these projects is a privacy tool. The cryptographic machinery enabling zk-Rollups was demonstrated to be practical by this 2014 paper. The proof system used in the paper, BCTV14, was later optimised to Groth16 in 2016.

	Zerocash (2014)	**ZK-Rollups (2020+)**	**zkEVM (2022+)**
Goal	Hide transactions	Scale transactions	Full Ethereum compatibility
Proof size	288 bytes (constant)	Constant	Constant
Key insight	Succinctness for privacy	Succinctness for scale	Succinctness for EVM

Table 2: The same core insight, repurposed across a decade of blockchain development.

5.3.3 The Regulatory Earthquake

Strong privacy on a public financial network was always going to be controversial. What this paper did was move that controversy from the theoretical world into the practical, and regulators were paying attention. The Financial Action Task Force (FATF), an intergovernmental institution that sets international standards to combat money laundering and terrorist financing, published guidance on cryptocurrency in 2019 that targeted what the institution calls "anonymity-enhanced cryptocurrencies" (AECs). FATF mostly described AECs as cryptocurrencies that hide the sender, receiver, and amount of the transaction, exactly the case made by Zerocash. Regulated exchanges responded quickly. Japan banned privacy coins in 2018, South Korea in 2019, and Australia in 2020. Major exchanges like Bittrex and ShapeShift delisted Zcash and other privacy coins under regulatory pressure. This left developers with a long-standing choice: favour privacy and potentially be excluded from regulated markets, or comply with regulation and sacrifice anonymity for access to those markets. That tension still exists today, and it began with this paper's attempt to make strong privacy technically credible.

The most striking thing is that the authors seem to have anticipated this and sketched a solution. At the end of the paper they note that zero-knowledge proofs are sufficiently flexible to allow a user to prove, for example to a regulator, that they paid taxes or that a transaction was under a legal limit, without revealing anything else. The question of whether privacy and accountability are necessarily opposed is the central challenge of projects like Aleo and the

Aztec Network, which are trying to establish programmable privacy networks that also enable financial accountability. Zerocash did not solve the problem, but it was the first to articulate it clearly. The authors saw the regulatory friction coming a decade earlier than many regulators, and pointed to a solution that the industry is still building.

6 Future Work and Suggested Improvements

The paper itself proposes three categories of improvement (Section 8 of the original paper), and subsequent practical work has extended these.

1. **Everlasting anonymity.** By restricting each address to a single use and replacing the on-ledger ciphertext with an out-of-band communication channel, the pour transaction becomes statistically hiding rather than only computationally hiding. This neutralises the quantum-adversary threat at the cost of additional infrastructure for sender–recipient communication.
2. **Fast block propagation.** A node can validate the proof-of-work and forward a block immediately, deferring Pour-transaction verification until the block's transactions are actually used downstream. This decouples propagation latency from SNARK verification cost. The trade-off is increased exposure to invalid blocks, mitigated in practice by the cost of forging proof-of-work.
3. **Storage mitigation.** CMList storage can be reduced to a per-block Merkle path of roughly 2 KiB. SNList growth can be mitigated by partitioning the list into sublists indexed by interval Merkle trees, with timestamped coins so that users do not need to maintain authentication paths against the entire history.
4. **Beyond the paper.** Real-world deployment in Zcash addressed the trusted setup risk through multi-party computation ceremonies in which any single honest participant suffices for security. Subsequent zk-SNARK constructions (e.g., Halo, Groth16 in tandem with universal trusted setups, and STARKs) have removed or weakened the trusted-setup assumption entirely, at varying costs in proof size and verification time. Hardware acceleration has also brought prover times down by one to two orders of magnitude since 2014.

7 Conclusion

Compared to Bitcoin and Zerocoin, Zerocash is far more privacy-enabling, allowing a high degree of privacy in a decentralised payment system. While Bitcoin exposes the details of transactions in a public ledger, and Zerocoin only partially addressed anonymity, Zerocash proposes a Decentralized Anonymous Payment scheme that hides sender, recipient, and amount using zk-SNARKs.

One of the strong points of the paper is that formal cryptographic definitions are combined with a practical construction. Mint and pour transactions, combined with zero-knowledge proofs, allow verifying transactions without disclosing sensitive information. Experimental findings also demonstrate significant improvements in efficiency compared to Zerocoin, with small proof sizes and quick verification, making the system viable at scale.

Zerocash still has its limitations, such as the requirement for a trusted setup, high proving costs, and exposure to network-level privacy leaks; but overall, the paper demonstrates that fully private decentralised payments are achievable and lays the foundation for future privacy-focused blockchain systems.

Beyond its immediate technical contribution, Zerocash showed that succinct zero-knowledge proofs can be made practical for general use, a result whose influence now extends far beyond anonymous payments. The same cryptographic machinery underpins modern blockchain scaling efforts such as zk-Rollups, powers programmable-privacy networks under active development, and continues to shape the conversation between researchers, regulators, and industry on how privacy and accountability can coexist on public ledgers. The paper therefore stands not merely as the foundation of a single cryptocurrency but as a turning point in how the field thinks about verifiable computation at scale.

Appendix: Individual Contributions

Name	Section Covered
Massimo Ridella	Introduction & Related Work
Gloria Paraschivoiu	Technical Contribution & Methodology
Andrei Baraitaru	Integration with Existing Ledger-based Currencies & Conclusion
Zaid Alsaheb	Results and Interpretation, Limitations and Improvements
Clara Mouzannar	Impact on Blockchain Evolution

Table 3: Individual contributions of Group 4 members.

References

Ben-Sasson, E., Chiesa, A., Garman, C., Green, M., Miers, I., Tromer, E., & Virza, M. (2014, May). *Zerocash: Decentralized Anonymous Payments from Bitcoin (extended version)*.

Camenisch, J., Hohenberger, S., & Lysyanskaya, A. (2005). Compact E-Cash. *Advances in Cryptology – EUROCRYPT 2005*, 302–321.

Chaum, D. (1982). Blind Signatures for Untraceable Payments. *Advances in Cryptology, Proceedings of CRYPTO '82*, 199–203.

Miers, I., Garman, C., Green, M., & Rubin, A. D. (2013). Zerocoin: Anonymous Distributed E-Cash from Bitcoin. *2013 IEEE Symposium on Security and Privacy*, 397–411.